Open Source Software Audit Project Report

Project Title: Open Source Software Audit and Shell Scripting Automation

Student Name: DIKSHA MITRA

Registration Number: 24BCE10138

Course: Open Source Software

Institution: VIT Bhopal University

Semester: Winter Semester 2025-26

Faculty: Dr. Adarsh Patel

Certificate

This is to certify that the project titled *"Open Source Software Audit and Shell Scripting Automation"* submitted by **DIKSHA MITRA** is a bonafide record of work carried out as part of the Open Source Software course at VIT Bhopal University.

Acknowledgement

I would like to express my sincere gratitude to my faculty members and the Open Source Software course instructors for providing me with the opportunity to work on this project. Their guidance and feedback helped me understand the practical aspects of open-source development and Linux shell scripting. I also thank the open-source community for making learning resources and tools freely available.

Abstract

This project report presents a comprehensive study and audit of open-source software along with the development of automation scripts using the Bash shell. The objective of this project is to understand the philosophy, licensing models, development workflow, and practical usage of open-source software while also gaining hands-on experience in Linux system interaction.

Git, a widely used distributed version control system, has been selected as the primary open-source software for analysis. In addition to theoretical understanding, the project includes the design and implementation of five Bash scripts that automate various system-level tasks such as system reporting, software inspection, directory permission auditing, log analysis, and interactive file generation.

Table of Contents

1. Introduction to Open Source Software
2. Evolution of Open Source Movement
3. Advantages and Challenges of Open Source
4. Overview of Selected Software: Git
5. Licensing and Legal Considerations
6. System Architecture and Working of Git
7. Development Environment and Tools Used
8. Fundamentals of Linux and Bash Shell
9. Shell Scripting Concepts Used in the Project
10. Detailed Description of Implemented Scripts
11. Testing Methodology and Results
12. Challenges Faced and Solutions
13. Security and Ethical Considerations
14. Learning Outcomes and Skills Developed
15. Future Enhancements
16. Conclusion
17. References
18. Appendix

Introduction to Open Source Software

Open-source software refers to computer programs whose source code is made publicly available so that anyone can view, study, modify, and distribute it. This concept is based on the principle that software should be freely accessible and that collaboration leads to better quality and innovation. Unlike proprietary software, where the source code is restricted and controlled by a company, open-source software encourages transparency and community participation.

The importance of open-source software has grown significantly over the past few decades. Many critical systems that power the internet, such as web servers, operating systems, and development tools, are built on open-source technologies. Examples include the Linux operating system, Apache web server, Python programming language, and Git version control system. These tools are maintained by large global communities that continuously improve them.

One of the core ideas behind open source is the concept of freedom. Users are free not only to use the software but also to understand how it works and modify it according to their needs. This makes open-source software highly customizable and adaptable to different environments. Organizations often prefer open-source solutions because they provide flexibility, reduce licensing costs, and avoid vendor lock-in.

Evolution of the Open Source Movement

The open-source movement has its roots in the early days of computing when software was often shared freely among researchers and developers. During the 1970s and 1980s, software began to be commercialized, and companies started restricting access to source code. This shift led to concerns among programmers who believed that software should remain a shared resource.

In response to these restrictions, Richard Stallman launched the GNU Project in 1983 with the goal of creating a completely free operating system. He also introduced the concept of free software and later helped establish the Free Software Foundation (FSF). The GNU General Public License (GPL) was developed to ensure that software would remain free and open for future users.

The term “open source” gained popularity in the late 1990s when Netscape released the source code of its web browser. This event demonstrated that open development could be successful even in commercial environments. Since then, open-source software has become a dominant force in the technology industry, with major companies like Google, Microsoft, and IBM actively contributing to open-source projects.

Advantages and Challenges of Open Source Software

Open-source software offers several advantages that make it attractive to both individuals and organizations. One of the biggest advantages is cost savings. Since most open-source software is available free of charge, it reduces the financial burden on students, developers, and small businesses. Additionally, open-source tools can be modified and redistributed, allowing organizations to customize them according to their specific needs.

Another major advantage is transparency. Because the source code is available to the public, it can be inspected for security vulnerabilities and bugs. This often results in more secure software because issues can be identified and fixed quickly by the community. The collaborative nature of open-source development also leads to rapid innovation and frequent updates.

However, open-source software also has some challenges. One of the main issues is the lack of formal support in some projects. While large projects like Linux and Git have strong communities, smaller projects may not receive regular updates or maintenance. Compatibility and usability can also be challenges, especially for users who are not familiar with technical configurations.

Overview of Selected Software: Git

Git is a distributed version control system designed to manage changes in source code during software development. It was created by Linus Torvalds in 2005 to support the development of the Linux kernel. Git allows multiple developers to work on the same project simultaneously while keeping track of every change made to the codebase.

Unlike centralized version control systems, Git allows every user to have a complete copy of the repository. This means that developers can work offline and still perform most version control operations. Git uses a snapshot-based approach rather than storing differences between files, which makes it fast and efficient.

Git has become the industry standard for version control due to its powerful features such as branching, merging, staging, and history tracking. Platforms like GitHub and GitLab provide hosting services that integrate Git with collaboration tools such as issue tracking and pull requests.

Licensing and Legal Considerations

Open-source software is governed by specific licenses that define how the software can be used, modified, and redistributed. These licenses ensure that while the software remains freely accessible, certain legal protections are maintained for the original authors and contributors. Understanding these licenses is essential because misuse or violation of license terms can lead to legal consequences, especially when software is reused in commercial or academic projects.

Git, the open-source software selected for this project, is distributed under the **GNU General Public License version 2 (GPLv2)**. This license follows the principle of *copyleft*, which means that any modified version of the software must also be released under the same license. This ensures that improvements made by developers remain open and accessible to the community. The GPL license promotes transparency and prevents proprietary entities from taking open-source code, modifying it, and distributing it as closed-source software.

Legal considerations in open-source development also include proper attribution, compliance with distribution terms, and ensuring that third-party code is used responsibly. Developers must carefully read and understand license terms before integrating open-source components into their projects. In academic environments, respecting software licenses is equally important, as it reflects ethical software usage and professional integrity. By studying Git's licensing model in this project, a better understanding of open-source legal frameworks and responsible software development practices was achieved.

System Architecture and Working of Git

Git is designed as a distributed version control system, which means that every user working on a project has a complete copy of the entire repository on their local machine. Unlike centralized systems that depend on a single server, Git allows developers to perform most operations such as commits, branching, and history viewing without requiring an internet connection. This architecture increases reliability, as work can continue even if the central server is temporarily unavailable.

Internally, Git does not store changes as simple differences between files. Instead, it stores data as a series of snapshots. Every time a commit is made, Git records the state of all tracked files at that particular moment. If files remain unchanged, Git simply references the previous version rather than duplicating the data. This snapshot-based storage model improves performance and ensures efficient use of storage space while maintaining a complete project history.

Git also uses a powerful data structure known as a directed acyclic graph (DAG) to manage commits and their relationships. Each commit points to its parent commit, forming a chain that represents the development timeline. Branches in Git are lightweight pointers that allow developers to work on different features or fixes simultaneously without affecting the main codebase. This flexible architecture makes Git highly efficient for collaborative development, enabling teams to merge changes, resolve conflicts, and maintain a clear history of project evolution.

Development Environment and Tools Used

The development of this project was carried out in a Linux-based command-line environment to ensure compatibility with open-source tools and shell scripting features. Since the system used for development was based on Windows, Git Bash was installed to simulate a Unix-like terminal environment. Git Bash provides access to common Linux commands and utilities, making it suitable for writing and testing Bash scripts without requiring a separate Linux installation.

Several open-source tools were used throughout the project to support development, testing, and version control. Git was used as the primary version control system to track changes in scripts and maintain different versions of the project files. The scripts were written and executed using the Bash shell, which allowed the use of features such as loops, conditional statements, command substitution, and file redirection. These tools made it possible to automate system-level tasks efficiently.

GitHub was used as the remote repository hosting platform to store and manage the project online. It provided features such as repository version history, commit tracking, and easy sharing of project files. Using GitHub also helped in understanding real-world workflows used in open-source development, such as committing changes, pushing updates, and maintaining a structured project repository. Overall, the selected development environment and tools ensured that the project remained fully aligned with open-source practices and industry standards.

Fundamentals of Linux and Bash Shell

Linux is a widely used open-source operating system known for its stability, security, and flexibility. It is based on the Unix architecture and is commonly used in servers, embedded systems, and development environments. One of the key strengths of Linux is its powerful command-line interface, which allows users to interact directly with the operating system and perform tasks efficiently using textual commands rather than graphical interfaces.

The Bash shell, which stands for Bourne Again Shell, is the default command-line interpreter in most Linux distributions. It acts as an interface between the user and the operating system kernel, allowing users to execute commands, manage files, and run programs. Bash supports various features such as command history, tab completion, scripting capabilities, and environment variables, making it a versatile tool for both beginners and advanced users.

In addition to executing individual commands, Bash allows users to write shell scripts, which are sequences of commands stored in a file and executed together. These scripts can automate repetitive tasks such as file management, system monitoring, and software installation. Understanding the fundamentals of Linux commands and Bash scripting was essential for completing this project, as all scripts were designed to run in a Unix-like environment and relied on core Linux utilities.

Shell Scripting Concepts Used in the Project

Shell scripting played a central role in this project, as it was used to automate system-related tasks and demonstrate interaction with the Linux command-line environment. Bash scripts are essentially text files containing a sequence of commands that are executed in order by the shell. These scripts help in reducing manual effort and improving efficiency by allowing repetitive operations to be performed automatically.

Several fundamental scripting concepts were applied during the development of the scripts. Variables were used to store and reuse data such as file names, user input, and command outputs. Conditional statements, including if, elif, and else, were used to make decisions within the scripts based on different conditions. Loops such as for and while were used to iterate over files, directories, or lines in a file, which made it possible to process large amounts of data systematically.

The project also made use of user input handling through the read command, allowing scripts to interact with users and accept dynamic values at runtime. Command-line arguments were used in some scripts to provide flexibility by allowing users to pass parameters while executing the script. Additionally, file handling techniques such as input and output redirection were used to read from and write to files. These concepts together formed the foundation of all the scripts developed in this project and helped in demonstrating practical automation using Bash.

Detailed Description of Implemented Scripts

This project includes five Bash scripts designed to demonstrate practical automation and system interaction in a Linux environment. Each script focuses on a different aspect of shell scripting, including system information retrieval, software inspection, file permission analysis, log processing, and interactive file generation. The following subsections describe each script in detail, including its purpose, working mechanism, and key commands used.

10.1 Script 1 – System Identity Report

The first script was developed to generate a concise report containing essential information about the current system. The script retrieves and displays details such as the current logged-in user, hostname, operating system kernel version, system uptime, and current date and time. This script demonstrates how system-level commands can be combined to produce a structured output useful for quick diagnostics.

The script makes use of standard Linux commands such as whoami, hostname, uname -r, uptime, and date. Command substitution is used to capture the output of these commands and display them in a formatted manner. This script highlights how shell scripting can be used to gather system information dynamically without requiring manual execution of multiple commands.

10.2 Script 2 – FOSS Package Inspector

The second script focuses on inspecting the presence and version of a specific open-source software package, namely Git. The purpose of this script is to verify whether Git is installed on the system and to display its version information if available. This script is useful for system auditing and ensuring that required tools are installed before starting development tasks.

The script uses conditional statements to check the availability of the git command using utilities such as which or direct command execution. If Git is installed, the script retrieves version details using the git --version command. If not, it displays an appropriate message informing the user that the software is missing. This script demonstrates the use of if-else conditions, error handling, and command output processing.

10.3 Script 3 – Disk and Permission Auditor

The third script was designed to examine the permissions and ownership details of selected directories in the system. File permissions are an important aspect of Linux security, as they

determine which users can read, write, or execute files. This script helps in understanding and auditing these permissions for critical directories.

The script uses a loop to iterate through a predefined list of directories. For each directory, the script checks whether it exists and then uses the ls -ld command to display its permission structure and ownership information. By automating this process, the script eliminates the need for manually checking each directory and ensures consistency in auditing system permissions.

10.4 Script 4 – Log File Analyzer

The fourth script was developed to analyze log files and count the occurrences of a specific keyword, such as “error” or “warning.” Log analysis is an important task in system administration and debugging because log files contain records of system events, application activities, and potential issues.

This script accepts a log file as input and reads it line by line using a while read loop. For each line, the script checks whether the specified keyword is present. If a match is found, a counter variable is incremented. After processing the entire file, the script displays the total number of occurrences. This script demonstrates file handling, pattern matching, loop structures, and the use of command-line arguments for flexible execution.

10.5 Script 5 – Open Source Manifesto Generator

The fifth script introduces interactive scripting by collecting input from the user and generating a personalized text file. The purpose of this script is to demonstrate how Bash scripts can be used not only for system tasks but also for generating user-defined content dynamically.

The script prompts the user with a series of questions related to their views on open-source software, such as why they support open source and how they plan to use it in the future. The responses are captured using the read command and then written into a text file using output redirection. The script also includes the current date in the generated file, making it a simple but effective example of file generation and user interaction in shell scripting.

Testing Methodology and Results

A systematic testing approach was followed to ensure that all the developed shell scripts functioned correctly under different conditions. Each script was executed multiple times in a Bash environment to verify that it produced the expected output and handled errors gracefully. Testing was performed using Git Bash on a Windows system, which provided a Linux-like terminal environment suitable for executing and debugging Bash scripts.

Different input scenarios were used during testing to evaluate the robustness of the scripts. For example, the log file analyzer script was tested with log files containing varying numbers of error messages to confirm that the counting mechanism worked accurately. Similarly, the script that checks for Git installation was tested on systems both with and without Git installed to verify that it correctly handled both situations. This helped in identifying logical errors and ensuring that the scripts behaved correctly under normal as well as edge-case conditions.

The results of the testing process showed that all scripts executed successfully and produced the intended outputs. Minor issues encountered during initial testing, such as incorrect file paths and improper handling of command-line arguments, were identified and resolved through debugging and code refinement. Screenshots of the successful execution of each script were captured and included in the project repository as evidence of correct functionality and to support the validation of results.

Challenges Faced and Solutions

During the development of this project, several challenges were encountered, particularly while working with shell scripting and command-line environments. One of the initial difficulties was understanding the syntax and structure of Bash scripts, especially the correct use of loops, conditional statements, and command substitution. Since shell scripting is sensitive to small errors such as missing spaces or incorrect quotation marks, even minor mistakes often caused scripts to fail or produce unexpected output.

Another challenge involved handling file paths and command-line arguments. Some scripts initially failed to execute properly because they were unable to locate input files or directories. This was especially evident in the log file analyzer script, where incorrect or missing file paths resulted in errors. To resolve this issue, additional validation checks were added to ensure that input files existed before processing them. This improved the reliability of the scripts and prevented runtime errors.

Debugging and testing also required careful attention, as error messages in Bash are sometimes not very descriptive. To overcome this, scripts were tested step by step, and intermediate outputs were printed to verify that each section of the script was working correctly. Online documentation and manual pages were referred to in order to understand the correct usage of commands and scripting constructs. By systematically identifying problems and applying appropriate fixes, the final versions of all scripts were made stable and functionally correct.

Security and Ethical Considerations

Security is an important aspect when working with open-source software and system-level scripts. Since shell scripts have the ability to access and modify system files, they must be written carefully to avoid unintended changes or security vulnerabilities. In this project, all scripts were designed to perform read-only operations or controlled file creation, ensuring that no critical system configurations were altered during execution. Proper testing was carried out before running scripts on important directories to prevent accidental data loss.

Ethical considerations are equally significant in the context of open-source development. Open-source software is built on trust, collaboration, and respect for community contributions. While using Git and other open-source tools in this project, care was taken to follow licensing requirements and avoid unauthorized redistribution of proprietary content. All scripts developed as part of this project were written independently and were not copied from external sources, ensuring academic integrity and originality of work.

Another important ethical aspect is responsible use of system information and logs. Log files and system reports may contain sensitive information such as user names, system paths, or configuration details. Therefore, scripts that analyze or display such data must be used only in controlled environments and for legitimate purposes such as learning, debugging, or system administration. By maintaining responsible practices and respecting software licenses, the project adheres to both technical and ethical standards expected in professional software development.

Learning Outcomes and Skills Developed

The completion of this project significantly improved my understanding of open-source software and its role in modern computing. By studying Git as a real-world open-source project, I gained insights into how large-scale software is developed, maintained, and distributed by a global community of contributors. This helped in understanding the importance of transparency, collaboration, and licensing in open-source development.

The project also provided practical experience in working with the Linux command-line environment and Bash shell scripting. Writing and executing scripts helped in developing a clear understanding of scripting concepts such as variables, loops, conditional statements, command substitution, and file handling. These skills are essential for automating repetitive tasks, managing systems efficiently, and working in professional development environments where command-line tools are widely used.

In addition to technical skills, the project enhanced problem-solving and debugging abilities. Encountering errors during script execution required careful analysis of command outputs and script logic. By identifying and resolving these issues, I developed a more systematic approach to troubleshooting and testing software. Overall, the project contributed to both theoretical knowledge and hands-on experience, preparing me for future work in software development, system administration, and open-source collaboration.

Future Enhancements

Although the current project successfully demonstrates the use of open-source tools and basic shell scripting, there are several areas where the system can be further enhanced. One possible improvement is the addition of more advanced scripts that perform real-time system monitoring, automated backups, and scheduled task execution using cron jobs. These features would make the project more aligned with real-world system administration practices and increase its practical applicability.

Another potential enhancement is improving the user interface and usability of the scripts. Currently, the scripts operate through command-line interaction, which is efficient but may be challenging for users who are not familiar with terminal-based environments. Future versions of the project could include a menu-driven script or a simple graphical interface that allows users to select different operations without needing to remember command syntax. This would make the system more user-friendly and accessible to beginners.

The project can also be extended by integrating additional open-source tools and expanding compatibility with different Linux distributions. For example, the scripts could be modified to automatically detect the operating system and adjust commands accordingly. Integration with cloud-based repositories and automated documentation generation are other areas that could be explored. These enhancements would help transform the project from a basic academic exercise into a more comprehensive and scalable open-source automation toolkit.

Conclusion

This project successfully combined the theoretical study of open-source software with practical implementation through shell scripting. By selecting Git as the subject of analysis, a deeper understanding was gained of how open-source tools are developed, licensed, and maintained by a global community. The study highlighted the importance of transparency, collaboration, and version control in modern software development practices.

The implementation of multiple Bash scripts provided hands-on experience in interacting with a Linux-like environment and automating common system tasks. Through the development of scripts for system reporting, software inspection, permission auditing, log analysis, and interactive file generation, the project demonstrated how shell scripting can simplify complex and repetitive operations. This practical exposure helped in strengthening both programming logic and familiarity with command-line tools.

Overall, the project served as a valuable learning experience that bridged the gap between theoretical concepts and real-world applications. It improved technical skills in Linux, Git, and scripting while also reinforcing awareness of open-source ethics and licensing. The knowledge and experience gained from this project will be beneficial for future academic work as well as professional roles in software development, system administration, and open-source contribution.

References

1. Git Official Documentation.
   Available at: https://git-scm.com/docs
   Accessed for understanding Git architecture, commands, and workflow.

2. Free Software Foundation.
   GNU General Public License Version 2 (GPLv2) Documentation.
   Available at: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
   Used to study licensing and legal considerations of open-source software.

3. GNU Bash Reference Manual.
   Available at: https://www.gnu.org/software/bash/manual/
   Used for learning Bash scripting syntax, commands, and shell behavior.

4. Linux Manual Pages and Command Documentation.
   Available through the man command in terminal (e.g., man ls, man grep, man bash).
   Used to understand Linux command-line utilities used in script development.

5. GitHub Documentation and Guides.
   Available at: https://docs.github.com/
   Used to understand repository management, commits, and version control workflows.

Appendix

The appendix section contains supplementary material that supports the implementation and execution of the project. This includes the source code of all Bash scripts developed during the project, along with sample outputs and screenshots demonstrating successful execution. The appendix is provided to give a clearer understanding of how the scripts were written and how they behave when executed in a Linux-based environment.

A. List of Developed Scripts

The following five scripts were created as part of this project:

- **script1.sh** – System Identity Report
  Displays system information such as username, hostname, kernel version, uptime, and date.
- **script2.sh** – FOSS Package Inspector
  Checks whether Git is installed on the system and displays version information.
- **script3.sh** – Disk and Permission Auditor
  Displays permission and ownership details of selected directories.
- **script4.sh** – Log File Analyzer
  Reads a log file and counts occurrences of a specified keyword such as “error”.
- **script5.sh** – Open Source Manifesto Generator
  Collects user input and generates a personalized text file describing views on open-source software.

B. Sample Script Execution

Each script was executed in a Bash terminal environment, and the outputs were verified for correctness. Screenshots of the script execution were captured and stored in the project repository under the **screenshots/** directory. These screenshots serve as evidence that the scripts run successfully without errors.